Week 6 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download Week 6 code

207 · Staged hints

LeetCode 207: Course Schedule · Medium

H1. Which courses have no remaining prerequisites?

H2. Can checking one component detect all cycles?

H3. Direct edges before→after, remove zero-indegree nodes, and compare the removed count with V.

207 · Correctness and cost

Each removed node has no remaining prerequisite. Every nonempty DAG has a zero-indegree node, so an unprocessed remainder must contain a cycle.

V is courses, E constraints. O(V+E) time and auxiliary space including the graph.

207 · Counterexamples and checks

[[1,0],[0,1]] is cyclic. A vertex in state 1 lies on the current search path. An edge back to it identifies a cycle. A vertex in state 2 has completed its search, so its result can be reused.

```
n=3, prerequisites=[[1,0],[2,1]] → True
```

Predict the state and return value before running this example. Then check the boundary cases above.

207 · Implementation

```
def can_finish(n, prerequisites):
    return len(course_order(n, prerequisites)) == n
```

210 · Staged hints

LeetCode 210: Course Schedule II · Medium

H1. Must the valid order be unique?

H2. How can the returned order be checked independently?

H3. Record Kahn queue removals and require V nodes at the end.

210 · Correctness and cost

A course enters only after all prerequisites were removed, so every before precedes after in the result. A short result cannot be a complete order.

V is courses, E edges. O(V+E) time and total graph/queue/indegree/output space.

210 · Counterexamples and checks

Reading [[1,0]] as 1→0 reverses the dependency. Validate edges instead of comparing with one chosen valid order.

```
n=4, prerequisites=[[2,0],[2,1],[3,2]] → [0,1,2,3]
```

Predict the state and return value before running this example. Then check the boundary cases above.

210 · Implementation

```
def course_order(n, prerequisites):
    graph, indegree = [[] for _ in range(n)], [0] * n
    for after, before in prerequisites:
        graph[before].append(after)
        indegree[after] += 1
    queue = deque(v for v in range(n) if indegree[v] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == n else []
```

78 · Staged hints

LeetCode 78: Subsets · Medium

H1. Will later pops change a stored path?

H2. Must later choices return to earlier indices?

H3. Choose an index at least start, recurse from the next index, pop, and store path.copy() at each call.

78 · Correctness and cost

Every subset has one increasing index sequence. start permits exactly that sequence, and restoration returns the parent path to its prior state.

n is length. O(n·2^n) time/output including snapshots; O(n) path and call stack.

78 · Counterexamples and checks

result.append(path) stores one shared object; later pops can turn multiple results into empty lists.

`nums=[2,5] → [[],[2],[2,5],[5]]`

Predict the state and return value before running this example. Then check the boundary cases above.

78 · Implementation

```
def subsets(nums):
    result, path = [], []

    def visit(start):
        result.append(path.copy())
        for index in range(start, len(nums)):
            path.append(nums[index])
            visit(index + 1)
            path.pop()

    visit(0)
    return result
```

39 · Staged hints

LeetCode 39: Combination Sum · Medium

H1. Which next start permits reusing the same candidate?

H2. Would remaining keep decreasing with nonpositive candidates?

H3. Sort candidates, recurse from the same index to permit reuse, and stop at values exceeding remaining.

39 · Correctness and cost

Positive choices decrease remaining and ensure termination. Nondecreasing indices avoid permutations while representing every combination.

m candidates,T target,c minimum,L=floor(T/c),B visited calls,S output elements. Time O(m log m+mB+S), auxiliary O(m+L), output O(S); conservatively B=O($(m+1)^{L+1}$).

39 · Counterexamples and checks

Recursing at index+1 misses [2,2] for [2], target=4. Zero is outside the contract and breaks termination.

```
candidates=[2,5], target=7 → [[2,5]]
```

Predict the state and return value before running this example. Then check the boundary cases above.

39 · Implementation

```
def combinations(candidates, target):
    values, result, path = sorted(candidates), [], []

    def visit(start, remaining):
        if remaining == 0:
            result.append(path.copy())
            return
        for index in range(start, len(values)):
            value = values[index]
            if value > remaining:
                break
            path.append(value)
            visit(index, remaining - value)
            path.pop()

    visit(0, target)
    return result
```

Execution and sources

```
cd code
python demo_11.py
python demo_12.py
python verify.py
```

Standard-library imports are in `solutions.py`. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Checkpoint answers and reasoning

1. None exists: each course waits for the other, so return `[]`.
2. Yes. Both 0 and 1 only need to precede 2.
3. Four subsets, including `[]`. Copy the path so later pops do not alter saved results.
4. Reuse index 0. Advancing to the next index misses `[2,2]`.

Concept-check explanations

DFS state 1 is on the active path, so an edge back to it identifies a cycle. State 2 has left that path. In `0→1`, `0→2`, `2→1`, revisiting completed node 1 is not a cycle.

A recursive return ends only that call. The parent resumes with pop and the next loop choice. Use Kahn's algorithm for long graphs to avoid recursion-depth limits.